

# What Transfers and What Does Not: Reproducing Compressed Sparse Attention at Single-GPU Scale

Brett Cleary

September 2026

## Abstract

We attempted to reproduce the central claim of Compressed Sparse Attention (CSA), the attention mechanism of DeepSeek-V4, at a scale reachable by one consumer GPU: roughly  $10^7$  parameters, byte-level modelling, and context lengths from 1K to 64K. We did not succeed, and the reasons are more informative than the intended result would have been.

Three findings concern measurement rather than architecture. First, the published equations under-determine the architecture: implementing Eqs. (9)–(19) faithfully omits a sliding-window branch that appears only in the accompanying figure, leaving every query blind to the  $t \bmod m$  bytes immediately preceding it, at a cost of roughly 0.52 bits per character. Second, our dense baseline lacked the query–key normalisation that CSA carries by specification; with a gradient-norm clip of 1.0, this left the baseline clipped on 94% of steps at 16K context and training at approximately one-fifteenth of its nominal learning rate, an artefact larger than the architecture effect under study. Third, mean bits-per-character—the standard metric—dilutes the long-context effect by roughly  $80\times$ : the median byte gains 0.0002 bpc from an additional 8K of context while the top 1% of bytes gain 1.83 bpc and account for 81% of the total.

The fourth finding concerns the efficiency claim itself, and splits it in two. CSA’s compute advantage did not survive reimplementations: against a dense baseline dispatching to a FlashAttention kernel, our CSA used  $4.3\times$  the memory and ran  $3.8\times$  slower at 8K, and exhausted memory at 16K on hardware where dense trained comfortably at 64K. Its KV-cache advantage, by contrast, reproduced exactly— $32\times$  at our configuration, invariant in context length—because it is a property of the architecture’s shape rather than of its kernels. We argue this distinction is the transferable lesson: architectural claims survive small-scale reproduction, engineering-dependent claims do not, and published descriptions rarely make clear which is which.

## 1 Introduction

Architecture research at frontier scale is not reproducible by independent researchers. A 1.6T-parameter model trained on 33T tokens is beyond verification outside a handful of laboratories. The implicit hope is that the *mechanisms* such work introduces are separable from the scale at which they were demonstrated: that one can implement an attention variant on a single GPU, observe a directionally similar benefit, and thereby verify the idea even if not the artefact.

This paper tests that hope on a specific case. Compressed Sparse Attention (CSA), introduced in the DeepSeek-V4 technical report [1], compresses the key–value cache by a factor  $m$  along the sequence dimension and then applies sparse top- $k$  selection over the compressed entries. It is presented as the enabling mechanism for million-token context. We implemented it from the paper’s equations, validated the implementation with unit tests, and set out to answer a single question:

*Does compressed sparse attention overtake dense attention as context length grows, at a scale a single GPU can reach?*

The answer is that the question is malformed, and discovering how it is malformed took four distinct experimental failures. We report them because each failure is a concrete, quantified obstacle that any small-scale reproduction of a long-context attention mechanism will encounter, and because three of the four were invisible without instrumentation we did not initially think to build.

### Contributions.

1. We show the published CSA equations under-determine the architecture, and quantify the cost of the resulting omission (Section 4.3).
2. We identify and measure a baseline-construction artefact—missing query-key normalisation—that exceeded the architecture effect being studied, and that grows with context length (Section 4.2).
3. We show that mean bits-per-character is the wrong instrument for long-context evaluation, and give a corruption-based measurement that recovers the effect it hides (Section 5).
4. We show that CSA’s compute advantage inverts without custom kernels while its KV-cache advantage reproduces exactly, and argue this is the general pattern (Section 6).

We explicitly do *not* report a clean quality comparison between CSA and dense attention. Section 4 explains why the comparisons we ran are uninterpretable, and we consider publishing them as though they were not to be the more serious error.

## 2 Background

### 2.1 Compressed Sparse Attention

CSA ([1], §2.3.1) proceeds in three stages. Given hidden states  $H \in \mathbb{R}^{n \times d}$ , it first forms two series of KV entries  $C^a, C^b \in \mathbb{R}^{n \times c}$  with compression weights  $Z^a, Z^b$ , and compresses each group of  $m$  tokens into a single entry  $C_i^{\text{Comp}} \in \mathbb{R}^c$  by a softmax-weighted sum over a  $2m$ -token window (Eqs. 9–12). Adjacent compressed entries overlap in the token ranges they draw from, so the sequence is compressed to  $n/m$  entries.

A *lightning indexer* then scores each compressed block against a query token through a low-rank query path shared with the core attention (Eqs. 13–17), and a top- $k$  selector retains only the highest-scoring blocks. Finally, core attention is performed in multi-query fashion [3], with every query head attending to the same retained compressed entries, which serve as both keys and values (Eqs. 18–19).

Two details matter for what follows. The paper applies RMSNorm [4] to queries and to compressed KV entries, stating that this “avoids exploding attention logits and may improve training stability” (§2.3.3). And it introduces an additional sliding-window attention branch, whose motivation we return to in Section 4.3.

### 2.2 What CSA claims

It is worth stating precisely, because it constrains what a reproduction can show. CSA is introduced “to enhance long-context *efficiency*”; the claim is cost, not quality. The training recipe makes this

| context   | sweep v2 |       | sweep v3 |       | sweep v3.1 |       |
|-----------|----------|-------|----------|-------|------------|-------|
|           | dense    | CSA   | dense    | CSA   | dense      | CSA   |
| 1K        | 1.375    | 1.968 | 1.338    | 1.903 | —          | —     |
| 2K        | 1.420    | 2.054 | 1.352    | 1.927 | 1.419      | 1.836 |
| 4K        | 1.565    | 2.443 | 1.506    | 2.048 | 1.643      | 1.934 |
| 8K        | 2.250    | 3.180 | 1.625    | 2.691 | 1.769      | 1.900 |
| gap at 1K | 0.593    |       | 0.565    |       | —          |       |
| gap at 8K | 0.930    |       | 1.066    |       | 0.131      |       |

**Table 1:** Test bpc for the same grid, run three times. The architecture and data are identical; only the iteration caps and stopping behaviour changed. The gap widens with context (v2), is flat then jumps (v3), or collapses toward crossover (v3.1). None of these are trustworthy; see Sections 4.1–4.3.

concrete: DeepSeek-V4 warms up with *dense* attention for the first 1T tokens and introduces sparse attention only at a sequence length of 64K, having trained at 4K and 16K densely (§4.2.2). CSA is never trained from scratch and never used below 64K.

This has a consequence that took us some time to internalise. Compression ( $m$  tokens to one entry) and selection (top- $k$  of those) are both lossy. At matched parameters, tokens and context, dense attention is an upper bound on what CSA can extract. Expecting CSA to *surpass* dense in bits-per-character is expecting a lossy approximation to beat what it approximates. The achievable target is parity at lower cost, and our framing question, quoted in Section 1, was malformed from the start in asking for more.

### 3 Experimental setup

All experiments use a decoder-only transformer with pre-norm RMSNorm [4], SwiGLU feed-forward blocks, learned absolute position embeddings, and a tied language-model head. Unless stated otherwise:  $d_{\text{model}} = 384$ , 6 layers, 6 heads (head dimension 64), giving 10.6M non-embedding parameters for the dense arm and 10.0M for CSA. CSA uses  $m = 4$ ,  $c = 64$ , and top- $k = n_{\text{blk}}/4$  (constant 25% sparsity). Optimisation is AdamW [11] at  $3 \times 10^{-4}$  decayed cosine to  $3 \times 10^{-5}$ , weight decay 0.1, gradient-norm clip 1.0, bf16 autocast, effective batch 32 sequences. Hardware is a single RTX 5070 Ti (16GB) for the measurements in Sections 5 and 6; the sweeps in Section 4 used rented H100 and A100 instances.

Data is byte-level. We use enwik8 [9] (90M/5M/5M train/validation/test) and PG-19 [10], the standard long-range language-modelling benchmark, subsampled to the same split sizes. All results are bits per character (equivalently bits per byte),  $\text{bpc} = \mathcal{L} / \ln 2$ , on the held-out test split, evaluated from the best-validation checkpoint. For PG-19, evaluation and training windows are document-aware: no window crosses a book boundary. This matters—on enwik8 at 16K context, 41% of corpus bytes lie in an article shorter than the window, against 0.1% for PG-19.

### 4 The intended experiment, and three reasons it failed

Our intended headline experiment was a sweep over context length  $\{1\text{K}, 2\text{K}, 4\text{K}, 8\text{K}\}$  crossed with  $\{\text{dense}, \text{CSA}\}$  at fixed parameters and token budget, looking for the context length at which the arms cross. We ran it three times. Table 1 shows the result.

| arm   | context | median grad. norm | steps clipped | effective LR            |
|-------|---------|-------------------|---------------|-------------------------|
| dense | 1K      | 0.52              | 10.8%         | $\times 1.00$           |
| dense | 2K      | 0.77              | 41.5%         | $\times 1.00$           |
| dense | 8K      | 10.69             | <b>91.0%</b>  | $\times \mathbf{0.094}$ |
| dense | 16K     | 15.42             | <b>94.0%</b>  | $\times \mathbf{0.065}$ |
| CSA   | 1K      | 0.58              | 0.3%          | $\times 1.00$           |
| CSA   | 2K      | 0.52              | 0.8%          | $\times 1.00$           |
| CSA   | 8K      | 0.62              | 17.6%         | $\times 1.00$           |

**Table 2:** Gradient norms by arm and context. The dense baseline’s gradients grow with context; CSA’s do not, because CSA normalises queries and keys by specification. At 8K and 16K the baseline takes steps roughly one tenth of their nominal size while CSA runs essentially unclipped.

Three sweeps produced three incompatible stories from identical architecture and data. We diagnosed three separate causes, and none of them is about attention.

#### 4.1 The learning-rate schedule was tied to the stopping criterion

Our cosine schedule annealed over `max_iters`, while early stopping halted runs before reaching it. A run that stops early therefore never completes its anneal, and two cells that stop at different fractions of their cap are not comparable. The cleanest demonstration is a pair of runs differing only in cap: identical seed, context, batch, and identical best step (2500), yet 1.625 bpc against 1.769 bpc—a difference of 0.144 bpc attributable entirely to the cosine horizon.

The bias was systematic rather than random. In sweep v3.1, every dense cell stopped at 20–31% of its schedule while every CSA cell ran to 51–93% of its own. The apparent gap collapse at 8K is what that asymmetry looks like. We now set the anneal horizon explicitly, independent of the stopping cap.

#### 4.2 Only one arm had query–key normalisation

The more serious error was in the baseline. Our CSA implementation applied RMSNorm to queries and compressed keys, because the paper specifies it. Our dense baseline applied none, going directly from the QKV projection into scaled dot-product attention. We had been careful to give both arms identical positional encoding for fairness; we did not think to check normalisation.

The consequence is Table 2. Gradient norms are recorded pre-clip, against a clip threshold of 1.0.

This artefact is both arm-asymmetric and context-dependent, so it bites hardest exactly where the claim under test lives. It plausibly accounts for the dense arm early-stopping so early at long context, for dense bpc *worsening* with context in all three sweeps, and for much of v3.1’s apparent gap collapse.

It became undeniable when a 16K run on PG-19 diverged outright: gradient norms grew from 0.3 to  $4.3 \times 10^5$ , best validation stranded at step 2100 of 10000, with everything after degrading by roughly 50%. Training and validation bpc moved together throughout, ruling out overfitting. Adding QK-normalisation to the baseline—standard practice for dense attention [5, 6], and what CSA already had—held gradient norms flat at 0.33–0.50 for all 10000 steps and improved best validation from 2.1876 to 1.4918 bpc.

That single normalisation layer was worth 0.70 bpc in the baseline. The architecture effect the study set out to measure spans 0.13–1.07 bpc. The baseline was the experiment.

### 4.3 The equations omit a component the figure includes

CSA’s causal mask permits a query at position  $t$  to attend only to compressed blocks covering positions  $[0, m\lfloor t/m \rfloor]$ . The  $t \bmod m$  tokens immediately preceding  $x_t$  are therefore unreachable through attention at *every* layer. We verified this with a gradient probe: for  $m = 4$ ,  $\partial \text{logits}[23]/\partial \text{embedding}[s]$  is exactly zero for  $s \in \{20, 21, 22\}$  at any depth, while the dense arm has no such gaps.

This is not a bug in our implementation of the equations. It is a property of the compressed branch, and the paper says so directly (§2.3.3):

“In order to strictly preserve causality in CSA and HCA, each query attends to only preceding compressed KV blocks. Consequently, a query cannot access information from other tokens within its own compressed block. ... For these reasons, we introduce a supplementary attention branch ... in a sliding window manner.”

The remedy is a sliding-window branch of  $n_{\text{win}}$  uncompressed entries, concatenated with the selected compressed entries before core attention. It appears in the CSA architecture figure, on the same page as the equations, as a concatenation node feeding the shared-KV attention. It does not appear in Eqs. (9)–(19): Eq. (19) takes keys and values to be the selected compressed entries alone.

An implementation built faithfully from the equations is therefore missing a component that the figure includes and that the text motivates explicitly. An order- $K$  byte  $n$ -gram proxy prices the resulting blind spot at +0.514 bpc averaged over  $t \bmod m$  (stable at +0.521 for  $K = 8$ ), which is comparable to the entire dense–CSA gap our sweeps reported at short context. We have since implemented the branch ( $n_{\text{win}}$  configurable; DeepSeek-V4 uses 128 at  $m = 4$ ).

We note that at our scale the paper’s constant is awkward:  $n_{\text{win}} = 128$  is 0.013% of a 1M context but 12.5% of a 1K context. A fixed architectural constant tuned for one regime does not transfer to another three orders of magnitude away, and at 1K it would supply a dense local window over an eighth of the sequence.

### 4.4 Consequence

The three artefacts do not share a direction. The learning-rate horizon and the QK-normalisation asymmetry both flatter CSA, and both worsen with context. The recency hole flattens dense. Each is comparable to or larger than the effect being measured. We therefore report no corrected version of Table 1: the correct conclusion is that these measurements are uninterpretable, not that they can be adjusted.

## 5 Mean bits-per-character is the wrong instrument

Having fixed the baseline, we asked a prior question: does context length change the outcome enough to be worth sweeping over? We trained a dense 16K model on PG-19 to completion (gradient norms flat, anneal completed, best validation at step 9700 of 10000) and measured test bpc bucketed by position within the evaluation window, since a token at position  $t$  has  $t$  bytes of available context.

The curve is nearly flat: 1.654 bpc over positions 0–1024, 1.546 over 1024–2048, then drifting slowly to 1.513 at 16K. A paired per-window test between an early slice (2–4K of context) and a late slice (14–16K) gives  $+0.0240 \pm 0.0086$  bpc ( $t = 2.8$ , 326 windows). Statistically detectable; practically negligible against architecture effects of 0.13–1.07 bpc.

We initially concluded that the context axis was inert and that the study was not viable on this data. That conclusion was wrong, and the error is instructive: mean bpc averages over every

| statistic                           | value                                  |
|-------------------------------------|----------------------------------------|
| mean gain from true distant context | +0.0226 bpc                            |
| <b>median gain</b>                  | <b>+0.0002 bpc</b>                     |
| tokens helped / hurt                | 54.2% / 45.8%                          |
| tokens gaining > 1 bit              | 0.96%, carrying 79% of all bits gained |
| top 1% of tokens                    | +1.83 bpc each, 81% of all bits gained |

**Table 3:** Value of genuine distant context, PG-19, 327,680 scored positions. Context before byte 8192 replaced with text from another book; only positions 12288 and beyond scored, so every scored token retains at least 4K of genuine local context. The mean agrees with the independent position-bucketed measurement (+0.024); the median does not.

position, while long-range dependence affects only a small subset of them. If 1% of bytes gain 2 bits and 99% gain nothing, the mean gain is 0.02 bpc—indistinguishable from no effect.

To measure the distribution instead, we score each token twice on sequences identical in length, absolute position, and local context, differing only in whether the *distant* context carries real information: bytes before a cutoff are replaced with prose from a different book. Table 3 gives the result.

The typical byte gains nothing from 8K of additional context—it is close to a coin flip whether it gains or loses. The effect is concentrated in roughly 1% of tokens gaining 1.83 bits each. Averaging dilutes a large effect by approximately  $80\times$ .

This reframes the evaluation problem. On the subpopulation that depends on distant retrieval, the effect is 1.83 bpc: larger than every artefact catalogued in Section 4, rather than far smaller. A long-context architecture comparison at this scale is feasible, but only conditioned on the tokens where long context matters.

**Two designs that failed.** Both produced apparent gains above 4 bpc—larger than the model’s entire bits-per-character, which is the signature to watch for. Masking attention to a short window at evaluation takes a model trained with full attention off its training distribution. Re-feeding only the recent bytes as their own sequence is worse: the sequence then contains no position-0 token, and transformers depend on an early-token attention sink [8], so the model collapses to near-random (5.70 bpc against 6.98 for a uniform guess over this vocabulary). Substituting real text from another document preserves length, positions, and the sink, and changes only information content.

## 6 The efficiency claim splits in two

CSA’s claim is efficiency, so we measured it directly rather than inferring it from FLOP counts. Table 4 reports peak memory and throughput for a forward, backward, and optimiser step at batch size 1.

The result inverts the premise. On this hardware CSA is strictly worse than dense on memory, on speed, and on maximum reachable context. The reason is that the comparison is not FLOPs against FLOPs. Dense attention dispatches to a FlashAttention kernel [7], which is  $O(N)$  in memory and never materialises the  $N \times N$  score matrix. Our CSA materialises an  $(b, n_h, n, n_{\text{blk}})$  score tensor, and autocast computes softmax in fp32, so a single 16K layer holds 1.5GB of attention probabilities. CSA’s  $4\times$  saving in attention FLOPs is swamped by roughly a  $15\times$  gap in kernel quality.

This is not a claim that CSA is inefficient. It is a claim that CSA’s efficiency is a property of the kernels DeepSeek built for it—the report describes a domain-specific language and mixed FP8/FP4

| context | dense (FlashAttention SDPA) |                  | CSA ( $m = 4$ , 25% top- $k$ ) |            |
|---------|-----------------------------|------------------|--------------------------------|------------|
|         | peak memory                 | throughput       | peak memory                    | throughput |
| 8K      | 1.53 GiB                    | 251K tok/s       | 6.65 GiB                       | 66K tok/s  |
| 16K     | 2.91 GiB                    | 176K tok/s       | out of memory                  |            |
| 32K     | 5.62 GiB                    | 110K tok/s       | out of memory                  |            |
| 64K     | <b>11.06 GiB</b>            | <b>67K tok/s</b> | out of memory                  |            |

**Table 4:** Training feasibility on one RTX 5070 Ti (13.6 GiB free). Dense trains at 64K context; our CSA cannot reach 16K, and at 8K uses  $4.3\times$  the memory while running  $3.8\times$  slower.

storage for exactly this purpose—and that those kernels, not the equations, carry the advantage. A reimplementaion of the mathematics reproduces none of it.

## 6.1 What does reproduce

The KV-cache claim behaves differently. Dense attention must store  $K$  and  $V$  for every past token and every head:  $2n_h d_h$  values per token per layer. CSA stores one compressed entry per  $m$  tokens, shared across all heads by multi-query attention, plus the indexer keys. For our configuration this is 9216 bytes per token against 288, a  $32\times$  reduction, invariant in context length (Table 5).

| context | dense KV cache | CSA KV cache | reduction  |
|---------|----------------|--------------|------------|
| 16K     | 144.0 MB       | 4.6 MB       | $31\times$ |
| 64K     | 576.0 MB       | 18.1 MB      | $32\times$ |
| 256K    | 2304.0 MB      | 72.1 MB      | $32\times$ |
| 1M      | 9216.0 MB      | 288.1 MB     | $32\times$ |

**Table 5:** KV cache at inference. FlashAttention does not help dense here: it avoids materialising attention scores during compute, but autoregressive decoding must still store every past key and value.

This number requires no kernel engineering, no training, and no scale. It follows from head count,  $m$ ,  $c$ , and multi-query sharing, and it holds identically for a 10M-parameter model on a consumer GPU and for a 1.6T-parameter model on a cluster. It is consistent in kind with the report’s claim of approximately 2% of a GQA-8 baseline, which additionally interleaves a more aggressively compressed attention variant.

## 7 Discussion

**Architectural claims transfer; engineering claims do not.** The two halves of Section 6 are the paper’s central observation. CSA’s KV-cache reduction is arithmetic on the shape of the architecture, and it reproduced exactly at  $10^{-5}$  of the original scale. CSA’s throughput advantage is a property of fused kernels, and it did not merely fail to reproduce—it inverted. Both are presented in the source report as aspects of a single efficiency story. Nothing in the presentation distinguishes the claim a reader can verify with a calculator from the claim that requires a compiler team.

We suspect this generalises. Sparsity, compression, and cache-sharing mechanisms are typically justified by asymptotic FLOP or memory arguments, while their realised benefit depends on whether a competitive kernel exists. A dense baseline in 2026 has FlashAttention; a novel sparse mechanism usually does not, outside the laboratory that introduced it. Small-scale reproduction will



systematically understate such mechanisms unless the reproduction also reimplements the kernels, which is a different project with a different skill set.

**The baseline is the experiment.** Our largest measured artefact was not in the mechanism under study but in what we compared it against. A missing normalisation layer, standard in contemporary dense attention and specified for CSA, was worth 0.70 bpc and a  $15\times$  difference in effective learning rate at 16K context—exceeding the architecture effect. It was invisible in loss curves and surfaced only from logged gradient norms. We would now regard per-arm gradient-norm statistics and clip-rate statistics as mandatory reporting for any architecture comparison, on the grounds that a gradient clip converts an instability difference into a silent learning-rate difference.

**Specification by figure.** The sliding-window branch is specified in a figure and a prose paragraph but absent from the numbered equations. An implementer working from the equations, which is the natural reading order for a mechanism given in nineteen numbered steps, produces a model with a structural blind spot at every layer. We do not think this is unusual, and we suggest that reports of new mechanisms state explicitly which components are load-bearing for the reported results.

**Metric selection.** Mean bits-per-character is standard for character-level language modelling and is the wrong instrument for long-context questions. It dilutes the phenomenon by roughly  $80\times$  in our measurement. Any evaluation of a long-context mechanism that reports only aggregate perplexity or bpc is, on this evidence, underpowered by one to two orders of magnitude.

## 8 Limitations

We do not report a valid quality comparison between CSA and dense attention. The sweeps in Table 1 predate all three corrections; a corrected sweep, conditioned on the subpopulation identified in Section 5, remains to be run. All results use a single seed. Our models are approximately 10M non-embedding parameters at byte-level granularity with learned absolute position embeddings rather than rotary embeddings, so conclusions about what a model can extract from distant context are specific to that regime; a larger model, or one with better positional generalisation, may exploit long range differently.

The efficiency inversion in Table 4 is a statement about a straightforward PyTorch implementation, not about CSA. We did not attempt a fused kernel, and a competent one would change the comparison substantially. We regard establishing *how much* of the gap is recoverable without laboratory-scale engineering as the most valuable follow-up this work suggests.

## 9 Conclusion

We set out to find the smallest scale at which compressed sparse attention demonstrates a measurable advantage over dense attention, and instead found that at single-GPU scale the measurement is dominated by artefacts of the comparison rather than by the mechanism. A missing baseline normalisation exceeded the effect under study; a component present only in a figure cost roughly 0.52 bpc; the standard metric diluted the phenomenon by  $80\times$ ; and the efficiency claim inverted without custom kernels while the cache claim reproduced exactly.

The reproducibility question for frontier architecture research is therefore not simply one of compute. It is that published mechanisms bundle claims with very different reproduction requirements, and the bundling is invisible. The KV-cache argument in CSA is verifiable by anyone



with a calculator. The throughput argument is not verifiable without rebuilding the kernel stack. Both are presented as efficiency. Distinguishing them, we suggest, would do more for independent verification of architecture research than any amount of additional compute at our end.

**Code.** The implementation, all run logs, and the measurement scripts supporting each table are available at <https://github.com/BrettCleary/mini-deepseek-v4>.

## References

- [1] DeepSeek-AI. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. Technical report, 2026.
- [2] DeepSeek-AI. *DeepSeek-V3.2: DeepSeek Sparse Attention*. Technical report, 2025.
- [3] N. Shazeer. Fast transformer decoding: one write-head is all you need. *arXiv:1911.02150*, 2019.
- [4] B. Zhang and R. Sennrich. Root mean square layer normalization. *NeurIPS*, 2019.
- [5] A. Henry, P. Dachapally, S. Pawar, and Y. Chen. Query-key normalization for transformers. *Findings of EMNLP*, 2020.
- [6] M. Dehghani et al. Scaling vision transformers to 22 billion parameters. *ICML*, 2023.
- [7] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.
- [8] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. *ICLR*, 2024.
- [9] M. Hutter. The human knowledge compression prize, 2006. <http://prize.hutter1.net>
- [10] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap. Compressive transformers for long-range sequence modelling. *ICLR*, 2020.
- [11] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *ICLR*, 2019.